

Лабораторна робота № 2

Задача комівояжера: генетичний алгоритм

Виконав: Самусь Дем'ян Михайлович

Група: ММШІ-2

Мета та постановка задачі

Знайти найкоротший замкнений маршрут у повному неорієнтованому графі: відвідати кожне з 10 міст рівно один раз і повернутися до початкового міста. Використано синтетичну симетричну матрицю відстаней. Її ненульові елементи є цілими числами від 10 до 99 в умовних одиницях; діагональ дорівнює нулю. Матриця наведена у повному виводі програми та у вихідному коді.

Місто 0 зафіксовано як початок і кінець циклу. Хромосома містить лише перестановку міст від 1 до 9. Наприклад, хромосома

```
3 6 9 2 5 8 1 4 7
```

задає маршрут

```
0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
```

Оцінкою маршруту є сума ваг усіх ребер циклу, включно з переходами від міста 0 до першого міста та від останнього до 0. Для наведеного прикладу це $28 + 21 + 13 + 25 + 14 + 27 + 17 + 23 + 16 + 19 = 203$ умовні одиниці, де 28 є відстанню від міста 0 до міста 3, а 19 є відстанню від міста 7 назад до міста 0. Менше значення означає кращий маршрут.

Алгоритм і параметри

Параметр	Значення
Популяція	100 випадкових перестановок
Турнірний відбір	5% популяції: 5 учасників
Схрещування	Перевпорядкування сегмента, імовірність 1.0
Мутація	Обмін двох міст, імовірність 0.10 на нащадка
Елітизм	1 незмінний найкращий маршрут
Зупинка	50 поколінь без покращення або 500 поколінь
Seed	43
Статистика	Кожні 10 поколінь, початкове й кінцеве

Початкова популяція складається з незалежних випадкових перестановок; повторення дозволені. Для кожного батька незалежно обираються п'ять різних позицій популяції, з яких перемагає маршрут із найменшою довжиною. Якщо у турнір потрапили маршрути довжиною 268, 305, 241, 297 і 260, батьком стає маршрут довжиною 241, а решта четверо відсіюються. Один маршрут може виграти обидва турніри.

Для схрещування випадково обираються два різні індекси. Сегмент включає обидві межі й містить щонайменше два міста. Для першого нащадка міста сегмента першого батька розміщуються в порядку їх появи у другому батьку. Решта позицій не змінюється. Другий нащадок утворюється симетрично. Нехай батьки та вибраний сегмент такі:

Батько 1:	1	2		3	4	5		6	7	8	9
Батько 2:	4	7		2	9	1		5	8	3	6

Сегмент першого батька містить міста 3, 4 і 5. У другому батьку вони розташовані в порядку 4, 5, 3, тому саме цей порядок потрапляє в сегмент першого нащадка. Сегмент другого батька містить міста 2, 9 і 1, а в першому батьку вони йдуть у порядку 1, 2, 9. Нашадки виходять такі:

Нашадок 1:	1	2		4	5	3		6	7	8	9
Нашадок 2:	4	7		1	2	9		5	8	3	6

Поза сегментом обидва нащадки повністю повторюють своїх батьків, а всередині сегмента змінюється лише порядок тих самих міст. Тому кожне місто залишається в маршруті рівно один раз. Такий оператор відповідає прикладу з лекції. Якщо порядок уже збігається, схрещування може не змінити маршрут.

Мутація незалежно для кожного нащадка з імовірністю 0.10 обмінює два випадкові різні міста. Вона може відбутися після схрещування. Наприклад, обмін другої та сьомої позицій дає таку зміну:

До мутації:	3	6	9	2	5	8	1	4	7	довжина	203
Після мутації:	3	1	9	2	5	8	6	4	7	довжина	456

Перестановка знову лишається коректною, бо міста не зникають і не дублюються, а лише міняються місцями. Цей приклад показує й іншу властивість мутації: окремий випадковий обмін частіше псує маршрут, ніж покращує, і корисним він стає саме у поєднанні з відбором, який такі гірші варіанти відкидає.

Місто 0 не бере участі в операторах. До нового покоління копіюється одна найкраща особина, решта місць заповнюється нащадками; зайвий нащадок відкидається. Завдяки такому копіюванню найкращий знайдений маршрут не може загубитися через невдалі схрещування чи мутації у наступному поколінні.

Лічильник стагнації скидається лише за строго меншої довжини. Інший маршрут тієї самої довжини не є покращенням. Зберігається перший знайдений найкращий маршрут. Контрольні точки містять найкращу і середню довжину та п'ять найкращих особин із повтореннями. Однакові маршрути серед перших п'яти не означають, що вся популяція однакова.

Запуск і результати

З каталогу лабораторної виконати:

```
uv run main.py
```

Параметри змінюються константами у файлі `main.py`. Matplotlib встановлюється через uv. Графік автоматично зберігається поруч із програмою у `convergence.png`, замінюючи попередній файл, без відкриття вікна.

За seed 43 знайдено маршрут:

```
0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
```

Його довжина дорівнює **203**. Він уперше з'явився в поколінні **4**. Алгоритм завершив роботу після **54** поколінь через 50 поколінь без покращення.

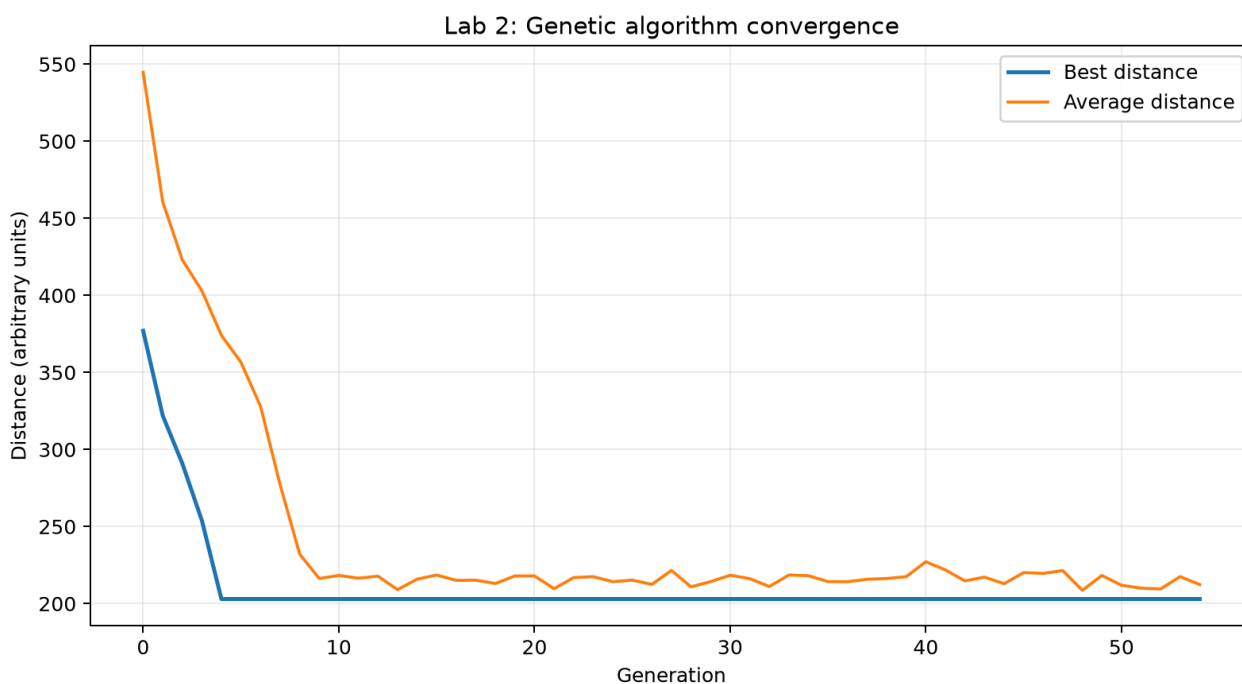


Рисунок 1: Найкраща та середня довжина маршруту в кожному поколінні.

Незалежна точна перевірка

Після запуску окремо перевірено всі $9! = 362\,880$ перестановок міст 1–9 із зафіксованим початком 0. Зворотні маршрути теж враховано; це не впливає на мінімум. Для кожного кандидата обчислено суму відстаней замкненого циклу. Мінімальна довжина дорівнює **203**, і такої довжини досягають рівно дві перестановки, які

є одна одній зворотними. Тобто з точністю до напрямку обходу оптимальний маршрут єдиний, і він збігається зі знайденим генетичним алгоритмом. Отже, розрив між результатом алгоритму та точним мінімумом нульовий.

Матрицю, метод, кількість перевірених перестановок та оптимальний маршрут збережено у `verification.json`. Точний перебір виконано заздалегідь: звичайний запуск програми його не виконує та не використовує результат для пошуку або зупинки. Після зміни матриці точну перевірку потрібно повторити. Для відтворення перевірки з каталогу `lab2` виконати:

```
uv run verify.py
```

Скрипт читає поточну матрицю з `main.py` та перезаписує `verification.json`. Повний код скрипта наведено в додатку Б.

Висновок

У ході виконання роботи реалізовано пошук найкоротшого замкненого маршруту задачі комівояжера генетичним алгоритмом на перестановочному кодуванні, де хромосома задає порядок обходу дев'яти міст, а місто 0 зафіксовано як початок і кінець циклу. Знайдено маршрут 0, 3, 6, 9, 2, 5, 8, 1, 4, 7, 0 довжиною 203 умовні одиниці.

Розібрано та запрограмовано всі стадії генетичного алгоритму: кодування маршруту перестановкою, обчислення довжини циклу, турнірний відбір батьків, схрещування перевпорядкуванням сегмента, мутацію обміном двох міст, елітизм і критерій зупинки за стагнацією. Роботу алгоритму продемонстровано на покроковому виведенні статистики та на графіку збіжності: найкраща довжина падає з 377 до 203 уже за чотири покоління, а середня по популяції підтягується до найкращої в міру збіжності.

Генетичний алгоритм є евристикою, тому зупинка за стагнацією не доводить оптимальності знайденого маршруту, а результат залежить від початкової популяції, випадкових операцій і обраних параметрів. Для цієї матриці збіг із точним мінімумом підтверджено окремим повним перебором.

Повний вивід програми

```
Lab 2: Travelling salesman problem with a genetic algorithm
Cities: 10 | Start/end city: 0 | Seed: 43
Population: 100 | Generation limit: 500
Tournament: 5% (5 contestants) | Elite: 1
Segment reordering crossover: 100%
Swap mutation per child: 10%
Stagnation limit: 50 | Log interval: 10
Distance matrix (arbitrary units):
  0   1   2   3   4   5   6   7   8   9
0:  0  34  71  28  95  46  63  19  82  57
1:  34   0  52  88  23  76  41  65  17  93
2:  71  52   0  36  69  14  85  47  91  25
3:  28  88  36   0  54  79  21  97  43  60
4:  95  23  69  54   0  32  74  16  58  81
5:  46  76  14  79  32   0  49  90  27  66
6:  63  41  85  21  74  49   0  38  72  13
7:  19  65  47  97  16  90  38   0  84  55
8:  82  17  91  43  58  27  72  84   0  31
9:  57  93  25  60  81  66  13  55  31   0
Generation  0 | Best: 377 | Average: 544.75
#1 | Distance: 377 | 0 -> 7 -> 9 -> 8 -> 1 -> 6 -> 5 -> 4 -> 2 -> 3 -> 0
#2 | Distance: 385 | 0 -> 3 -> 2 -> 5 -> 6 -> 1 -> 9 -> 8 -> 4 -> 7 -> 0
#3 | Distance: 390 | 0 -> 3 -> 2 -> 7 -> 9 -> 6 -> 5 -> 8 -> 1 -> 4 -> 0
#4 | Distance: 397 | 0 -> 6 -> 9 -> 2 -> 5 -> 8 -> 4 -> 7 -> 1 -> 3 -> 0
#5 | Distance: 397 | 0 -> 3 -> 8 -> 5 -> 2 -> 1 -> 6 -> 7 -> 4 -> 9 -> 0
Generation 10 | Best: 203 | Average: 218.27
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Generation 20 | Best: 203 | Average: 217.98
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Generation 30 | Best: 203 | Average: 218.43
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Generation 40 | Best: 203 | Average: 227.17
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Generation 50 | Best: 203 | Average: 211.90
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Generation 54 | Best: 203 | Average: 212.33
#1 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#2 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#3 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#4 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
#5 | Distance: 203 | 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0

Stopped after 54 generations: stagnation limit reached.
Best route: 0 -> 3 -> 6 -> 9 -> 2 -> 5 -> 8 -> 1 -> 4 -> 7 -> 0
Best distance: 203
First found at generation: 4
Chart saved: convergence.png
```

Додаток А. Повний код основної програми

```
import math
import random
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

POPULATION_SIZE = 100
MAX_GENERATIONS = 500
TOURNAMENT_FRACTION = 0.05
CROSSOVER_PROBABILITY = 1.0
MUTATION_PROBABILITY = 0.10
ELITE_COUNT = 1
STAGNATION_LIMIT = 50
RANDOM_SEED = 43
LOG_INTERVAL = 10
CHART_PATH = Path(__file__).with_name("convergence.png")

DISTANCES = [
    [0, 34, 71, 28, 95, 46, 63, 19, 82, 57],
    [34, 0, 52, 88, 23, 76, 41, 65, 17, 93],
    [71, 52, 0, 36, 69, 14, 85, 47, 91, 25],
    [28, 88, 36, 0, 54, 79, 21, 97, 43, 60],
    [95, 23, 69, 54, 0, 32, 74, 16, 58, 81],
    [46, 76, 14, 79, 32, 0, 49, 90, 27, 66],
    [63, 41, 85, 21, 74, 49, 0, 38, 72, 13],
    [19, 65, 47, 97, 16, 90, 38, 0, 84, 55],
    [82, 17, 91, 43, 58, 27, 72, 84, 0, 31],
    [57, 93, 25, 60, 81, 66, 13, 55, 31, 0],
]

def route_length(route: list[int]) -> int:
    cycle = [0, *route, 0]
    return sum(DISTANCES[a][b] for a, b in zip(cycle, cycle[1:]))

def format_route(route: list[int]) -> str:
    return " -> ".join(map(str, [0, *route, 0]))

def select_parent(
    population: list[list[int]], rng: random.Random
) -> list[int]:
    size = math.ceil(len(population) * TOURNAMENT_FRACTION)
    return min(rng.sample(population, size), key=route_length)

def crossover(
    parent1: list[int], parent2: list[int], rng: random.Random
) -> tuple[list[int], list[int]]:
    child1, child2 = parent1.copy(), parent2.copy()
    if rng.random() < Crossover_PROBABILITY:
        start, end = sorted(rng.sample(range(len(parent1)), 2))
        end += 1
        cities1 = set(parent1[start:end])
        cities2 = set(parent2[start:end])
        child1[start:end] = [city for city in parent2 if city in cities1]
        child2[start:end] = [city for city in parent1 if city in cities2]
    return child1, child2

def mutate(child: list[int], rng: random.Random) -> None:
    if rng.random() < MUTATION_PROBABILITY:
        a, b = rng.sample(range(len(child)), 2)
        child[a], child[b] = child[b], child[a]

def print_statistics(generation: int, population: list[list[int]]) -> None:
    scores = [route_length(route) for route in population]
    print(
        f"Generation {generation:4d} | Best: {min(scores):4d}"
        f" | Average: {sum(scores) / len(scores):8.2f}"
    )
    for rank, route in enumerate(sorted(population, key=route_length)[:5], 1):
```

```

        print(f" #[{rank}] | Distance: {route_length(route):4d}"
              f" | {format_route(route)}")

def validate_settings() -> None:
    for name, value in (
        ("POPULATION_SIZE", POPULATION_SIZE),
        ("MAX_GENERATIONS", MAX_GENERATIONS),
        ("ELITE_COUNT", ELITE_COUNT),
        ("STAGNATION_LIMIT", STAGNATION_LIMIT),
        ("LOG_INTERVAL", LOG_INTERVAL),
    ):
        if type(value) is not int or value < 1:
            raise ValueError(f"{name} must be a positive integer")
    if POPULATION_SIZE < 2:
        raise ValueError("POPULATION_SIZE must be at least 2")
    if ELITE_COUNT >= POPULATION_SIZE:
        raise ValueError("ELITE_COUNT must be smaller than POPULATION_SIZE")
    if not 0 < TOURNAMENT_FRACTION <= 1:
        raise ValueError("TOURNAMENT_FRACTION must be in (0, 1]")
    for name, value in (
        ("CROSSOVER_PROBABILITY", CROSSOVER_PROBABILITY),
        ("MUTATION_PROBABILITY", MUTATION_PROBABILITY),
    ):
        if not 0 <= value <= 1:
            raise ValueError(f"{name} must be in [0, 1]")
    size = len(DISTANCES)
    if size < 3 or any(len(row) != size for row in DISTANCES):
        raise ValueError("DISTANCES must be square with at least 3 cities")
    for i in range(size):
        for j in range(size):
            value = DISTANCES[i][j]
            if type(value) is not int:
                raise ValueError("Distances must be integers")
            if (i == j and value != 0) or (i != j and value <= 0):
                raise ValueError("Use zero diagonal and positive distances")
            if value != DISTANCES[j][i]:
                raise ValueError("DISTANCES must be symmetric")

def save_chart(history: list[tuple[int, float]]) -> None:
    fig, ax = plt.subplots(figsize=(9, 5))
    ax.plot([best for best, average in history], label="Best distance", linewidth=2)
    ax.plot([average for best, average in history], label="Average distance")
    ax.set(xlabel="Generation", ylabel="Distance (arbitrary units)",
           title="Lab 2: Genetic algorithm convergence")
    ax.grid(alpha=0.25)
    ax.legend()
    fig.tight_layout()
    fig.savefig(CHART_PATH, dpi=180)
    plt.close(fig)

def main() -> None:
    validate_settings()
    rng = random.Random(RANDOM_SEED)
    cities = list(range(1, len(DISTANCES)))
    population = [rng.sample(cities, len(cities)) for _ in range(POPULATION_SIZE)]
    best = min(population, key=route_length).copy()
    best_generation = 0
    stagnant_generations = 0
    history = []
    stop_reason = "generation limit reached"

    print("Lab 2: Travelling salesman problem with a genetic algorithm")
    print(f"Cities: {len(DISTANCES)} | Start/end city: 0 | Seed: {RANDOM_SEED}")
    print(f"Population: {POPULATION_SIZE} | Generation limit: {MAX_GENERATIONS}")
    print(f"Tournament: {TOURNAMENT_FRACTION:.0%}"
          f" | ({math.ceil(POPULATION_SIZE * TOURNAMENT_FRACTION)} contestants)"
          f" | Elite: {ELITE_COUNT}")
    print(f"Segment reordering crossover: {CROSSOVER_PROBABILITY:.0%}")
    print(f"Swap mutation per child: {MUTATION_PROBABILITY:.0%}")
    print(f"Stagnation limit: {STAGNATION_LIMIT} | Log interval: {LOG_INTERVAL}")
    print("Distance matrix (arbitrary units):")
    print("    " + " ".join(f"{city:3d}" for city in range(len(DISTANCES))))
    for city, row in enumerate(DISTANCES):
        print(f"{city:3d}: " + " ".join(f"{distance:3d}" for distance in row))

def record(population: list[list[int]]) -> None:

```

```

    scores = [route_length(route) for route in population]
    history.append((min(scores), sum(scores) / len(scores)))

record(population)
print_statistics(0, population)

generation = 0
for generation in range(1, MAX_GENERATIONS + 1):
    new_population = [best.copy() for _ in range(ELITE_COUNT)]
    while len(new_population) < POPULATION_SIZE:
        parents = [select_parent(population, rng) for _ in range(2)]
        for child in crossover(*parents, rng):
            if len(new_population) == POPULATION_SIZE:
                break
            mutate(child, rng)
            new_population.append(child)
    population = new_population

    candidate = min(population, key=route_length)
    if route_length(candidate) < route_length(best):
        best = candidate.copy()
        best_generation = generation
        stagnant_generations = 0
    else:
        stagnant_generations += 1

    record(population)
    stagnated = stagnant_generations >= STAGNATION_LIMIT
    finished = stagnated or generation == MAX_GENERATIONS
    if generation % LOG_INTERVAL == 0 or finished:
        print_statistics(generation, population)
    if finished:
        if stagnated:
            stop_reason = "stagnation limit reached"
            break

print(f"\nStopped after {generation} generations: {stop_reason}.")
print(f"Best route: {format_route(best)}")
print(f"Best distance: {route_length(best)}")
print(f"First found at generation: {best_generation}")
save_chart(history)
print(f"Chart saved: {CHART_PATH.name}")

if __name__ == "__main__":
    main()

```

Додаток Б. Скрипт точної перевірки

```
import json
from itertools import permutations
from pathlib import Path

from main import DISTANCES

OUTPUT_PATH = Path(__file__).with_name("verification.json")

def main() -> None:
    size = len(DISTANCES)
    if size < 3 or any(len(row) != size for row in DISTANCES):
        raise ValueError("DISTANCES must be square with at least 3 cities")
    for i, row in enumerate(DISTANCES):
        for j, value in enumerate(row):
            if type(value) is not int:
                raise ValueError("Distances must be integers")
            if (i == j and value != 0) or (i != j and value <= 0):
                raise ValueError("Use zero diagonal and positive distances")
            if value != DISTANCES[j][i]:
                raise ValueError("DISTANCES must be symmetric")

    best_route = None
    best_distance = None
    count = 0
    for route in permutations(range(1, size)):
        cycle = [0, *route, 0]
        distance = sum(
            DISTANCES[cycle[i]][cycle[i + 1]] for i in range(size)
        )
        count += 1
        if best_distance is None or distance < best_distance:
            best_distance = distance
            best_route = cycle

    result = {
        "method": "Exhaustive enumeration with city 0 fixed",
        "permutations": count,
        "matrix": DISTANCES,
        "optimal_route": best_route,
        "optimal_distance": best_distance,
    }
    OUTPUT_PATH.write_text(json.dumps(result, indent=2) + "\n", encoding="utf-8")
    print(f"Checked permutations: {count}")
    print("Optimal route: " + " -> ".join(map(str, best_route)))
    print(f"Optimal distance: {best_distance}")
    print(f"Verification saved: {OUTPUT_PATH.name}")

if __name__ == "__main__":
    main()
```